

Scripts d'extractions et de traitement de données de vie réelle d'ARIA à des fins de recherche clinique

ARIA ODM Builder

Notice visuelle du dépôt GitHub



Comment lire ce document

But du PDF

Ce document fonctionne comme une notice visuelle : il explique l'ordre du pipeline, les fichiers importants, les commandes et les sorties attendues.

Principe : suivre les étapes dans l'ordre et conserver les noms de fichiers.

Lecture rapide

1. Préparer le dossier et l'environnement
2. Vérifier la connexion SQL
3. Lancer l'extraction
4. Utiliser Streamlit et exporter

Extraire des données de vie réelle d'ARIA en cohortes de patients structurées par centre d'intérêt à des fins de recherche clinique.

Extraire

Récupérer les informations utiles avec les requêtes SQL.

Préparer

Générer les fichiers d'entrée CSV et Excel avec le script principal.

Exporter

Produire des résultats structurés avec l'application Streamlit.

Créer le dossier d'extraction SQL

À créer sur le poste

```
mkdir Extraction_SQL  
cd "chemin\vers\Extraction_SQL"
```

Le nom peut être adapté. Dans la suite, `nom_du_dossier` désigne le dossier créé pour l'extraction.

Outils possibles

Visual Studio Code, PyCharm, Windows Terminal ou PowerShell.

Condition : le terminal doit pointer vers le dossier créé.

Contenu attendu du dossier d'extraction

```
nom_du_dossier/  
|-- all_patient_ethos.sql  
|-- all_patient_formulaire.sql  
|-- all_patient_traitement.sql  
|-- load_final_all.py  
|-- requirements_sql.txt  
|-- sys.py  
|-- tox.py  
'-- trait.py
```

Point d'entrée

Le script à lancer est
load_final_all.py.

Dépendances

Le fichier à utiliser est
requirements_sql.txt.

Requêtes

Les fichiers .sql restent au même
niveau que le script.

Se placer dans le bon dossier

Depuis le terminal

```
cd "chemin\vers\nom_du_dossier"
```

Vérification simple

La commande `dir` doit afficher
`load_final_all.py`, les fichiers `.sql`
et `requirements_sql.txt`.

```
dir
```

Vérifier le pilote SQL Server disponible

Commande PowerShell

```
Get-OdbcDriver | Where-Object Name -like "*SQL Server*" | Select-Object Name,Platform
```

Driver

Noter le nom exact affiché.

Serveur / port

Vérifier les valeurs fournies par l'environnement local.

Base

Vérifier le nom exact de la base ARIA.

Ces informations seront reportées dans la fonction `connecter_bdd()` du fichier `load_final_all.py`.

Renseigner la connexion dans `connecter_bdd()`

```
self.connexion = pyodbc.connect(  
    "Driver={REPLIR_ICI_DRIVER};"  
    "Server=REPLIR_ICI_NOM_SERVEUR,REPLIR_ICI_PORT;"  
    "Database=REPLIR_ICI_NOM_BASE;"  
    "Trusted_Connection=yes;"  
    "TrustServerCertificate=yes;"  
)
```

Remplacer uniquement les valeurs
REPLIR_ICI.

Le champ Driver doit reprendre le nom
exact obtenu à la diapo précédente.

Créer l'environnement, installer, lancer

Commandes à exécuter

```
py -m venv .venv_sql  
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass  
.\.venv_sql\Scripts\Activate.ps1  
python -m pip install --upgrade pip  
python -m pip install -r requirements_sql.txt  
python .\load_final_all.py
```

Activation correcte

Le terminal affiche
(.venv_sql) au début de la
ligne.

À faire une fois

La création de l'environnement
et l'autorisation PowerShell
ne sont pas à refaire à chaque
lancement.

Sorties attendues après extraction

```
traitement_patient.csv  
traitement_patient.xlsx  
formulaire_patient.csv  
formulaire_patient.xlsx  
ethos_patient.csv  
ethos_patient.xlsx
```

Contrôle minimal

Vérifier la présence des six fichiers dans le dossier de travail.

Étape suivante

Ces fichiers deviennent les entrées de l'application Streamlit.

`samples/` sert uniquement si aucune base de données n'est disponible pour tester.

Dossier de l'application Streamlit

```
nom_du_dossier/  
|-- app.py  
|-- conf/  
|   |-- mapping.csv  
|   '-- profiles/  
|-- pictures/  
|-- samples/  
|   |-- csv/  
|   '-- xlsx/  
|-- utils/  
|-- requirements.txt  
'-- README.md
```

Point d'entrée

L'application se lance depuis le dossier qui contient `app.py`.

Configuration

`conf/` contient le mapping et les profils JSON.

Modules

`utils/` contient les fonctions utilisées par l'application.

Lancer l'application Streamlit

Commandes usuelles

```
cd "chemin\vers\nom_du_dossier"  
python -m pip install -r requirements.txt  
python -m streamlit cache clear  
python -m streamlit run app.py
```

L'interface s'ouvre dans le navigateur.

Charger les fichiers générés par l'extraction ou les fichiers de samples/ pour une démonstration sans base.

Streamlit - 1. Importer les fichiers

Objectif

Charger les fichiers nécessaires au calcul : traitement, formulaire, et éventuellement ETHOS.

Obligatoires : traitement_patient et formulaire_patient.

Optionnels : ethos_patient, mapping.csv, profil JSON.

samples/ peut servir uniquement pour une démonstration sans accès direct à la base.

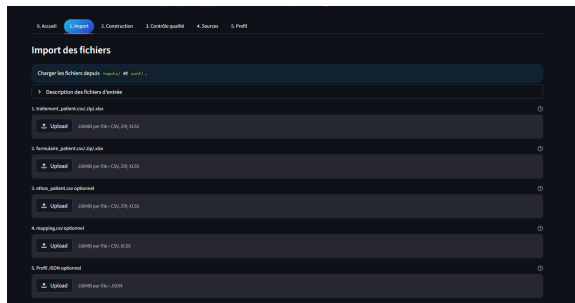


Figure – Onglet d'import

Streamlit - 2. Choisir CIM10 et mapping

Objectif

Définir la cohorte clinique : code CIM10, localisation ou centre d'intérêt.

Le fichier `mapping.csv` aide à retrouver les localisations et requêtes associées.

Le mode CIM10 permet de travailler sur le traitement considéré ou sur le patient global.

Vérifier la cohérence du CIM10 avant de construire l'export.

Aperçu mapping

	CIM10	Description	Requête 1	Requête 2	Requête 3	_client_mapping
0	C86.3	T.M. DE L'OREILLE MOY.	1 TETE requete			mapping.csv
1	C43.8	T.M. A LOC. CONTIGUES DES OS ET DU CARTILAGE ARTIC.	1 TETE requete			mapping.csv
2	C43.8	T.M. DES OS ET DU CARTILAGE ARTIC., SAI	1 TETE requete			mapping.csv
3	C43.2	MELANOME MALIN DE L'OREILLE ET DU CONDUIT AUDITIF EXT.	1 TETE requete			mapping.csv
4	C43.4	MELANOME MALIN DU CURR CHERILU ET DU COU	1 TETE requete			mapping.csv
5	C43.8	LES A LOC. CONTIGUES D'UN MELANOME MALIN DE LA PEAU	1 TETE requete			mapping.csv
6	C43.8	MELANOME DE LA PEAU SAI	1 TETE requete			mapping.csv
7	C44.2	T.M. DE LA PEAU DE L'OREILLE ET DU CONDUIT AUDITIF EXT.	1 TETE requete			mapping.csv
8	C44.4	T.M. DE LA PEAU DU CURR CHERILU ET DU COU	1 TETE requete			mapping.csv
9	C44.8	T.M. A LOC. CONTIGUES DE LA PEAU	1 TETE requete			mapping.csv

Figure – Aperçu du mapping

Streamlit - 3. Construire la sélection clinique

Objectif

Sélectionner les colonnes cliniques utiles à l'export patient.

Les colonnes proposées viennent du profil JSON, du mapping ou de la recherche manuelle.

Les compteurs indiquent les valeurs disponibles et le nombre de patients concernés.

Le rapport qualité est en cours de développement et reste secondaire pour l'instant.

Construction de l'export

Mode CM13: CM13 pour le traitement considéré | CM13 à inclure: Cdx | Dose non nulle | Référence début: Début RT / startDate

Aide mapping

Recherche libre dans mapping: prostate, sein, poumon, OBL...

CM13	Description	Source mapping	Localisations / requêtes proposées
Cdx	TJM DE LA PROSTATE	mapping.csv	TURBO REQUETE

Profil détecté: Cdx PROSTATE USD CSV

Figure – Construction de la cohorte

Streamlit - 4. Sélectionner les temporalités

Point clé

Dans le tableau, cocher pour chaque variable les temporalités à générer.

Cumul : toutes les valeurs disponibles pour la cohorte.

Avant RT : informations relevées avant le début de la radiothérapie.

Aigu / Pendant RT et Tardif / Après RT : suivi pendant et après traitement.

Nom report	Source selection	Catégorie	Pertinence	Score pertinence	Valeurs cohorte	Patients cohorte	☐ Cumul	☐ Avant RT	☐ Aigu / Pendant RT	☐ Tardif / Après RT
âge	Profil JSON	Clinique PROSTATE / URO	Très pertinent	82	6762	212	☒	☒	☒	☒
lignes relationnelles	Profil JSON	Clinique PROSTATE / URO	Très pertinent	82	6462	210	☒	☒	☒	☒
vélosité miclionnelle (urgence urinaire)	Profil JSON	Clinique PROSTATE / URO	Très pertinent	82	6347	210	☒	☒	☒	☒
nombre de lever nocturne	Profil JSON	Clinique PROSTATE / URO	Très pertinent	82	1128	206	☒	☒	☒	☒
ictile (rectal)	Profil JSON	Clinique PROSTATE / URO	Très pertinent	82	5668	206	☒	☒	☒	☒
lakturie	Profil JSON	Clinique PROSTATE / URO	Très pertinent	82	5557	207	☒	☒	☒	☐
nombre de protection urinaires (j)	Profil JSON	Clinique PROSTATE / URO	Très pertinent	82	1626	127	☒	☒	☒	☒
ictère total	Profil JSON	Clinique PROSTATE / URO	Très pertinent	82	40	8	☒	☒	☐	☒
S (International Prostate Symptom Score)	Profil JSON	Clinique PROSTATE / URO	Très pertinent	82	18	2	☒	☒	☐	☒
rythme	Profil JSON	Toxicité clinique	Très pertinent	88	6847	213	☒	☒	☒	☒
antenne urinaire	Profil JSON	Toxicité clinique	Très pertinent	88	1333	111	☒	☒	☐	☒
Risque de chute	Profil JSON	Toxicité clinique	Très pertinent	88	302	51	☒	☒	☐	☒

Figure – Sélectionner les temporalités

Streamlit - 5. Recalculer et exporter

Action finale

Cliquer sur **Construire / recalculer** après modification des colonnes ou des temporalités.

Exporter le résultat final en CSV ou Excel.

Télécharger le rapport de preuve Excel pour tracer les choix effectués.

Sauvegarder le profil JSON pour réutiliser la même sélection de colonnes et temporalités.



Figure – Enter Caption

Fonctionnalités principales de Streamlit

Import

Chargement des fichiers patient issus de l'extraction.

Mapping

Correspondance entre colonnes sources et variables du projet.

Temporalité

Analyse avant, pendant ou après radiothérapie.

CIM10 / localisation

Définition de la cohorte clinique à explorer.

Contrôles

Cohérence, dates, doublons et données manquantes.

JSON

Sauvegarde des choix pour éviter de refaire la même configuration.

Résumé opérationnel

1. Créer le dossier d'extraction et y placer les fichiers SQL, Python et `requirements_sql.txt`.
2. Vérifier le pilote ODBC, le serveur, le port et la base dans `connecter_bdd()`.
3. Créer ou activer `.venv_sql`, installer les dépendances et lancer `python .\load_final_all.py`.
4. Vérifier les six fichiers CSV/XLSX générés.
5. Lancer Streamlit depuis le dossier contenant `app.py`.
6. Importer, mapper, choisir CIM10/localisation, définir la période RT, contrôler, exporter et sauvegarder le JSON.